

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №2**  
**по дисциплине «Построение и анализ алгоритмов»**  
**Тема: кратчайшие пути в графе: коммивояжёр**  
**Вариант 8**

Студент гр. 4343

Васильев А. С.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

### **Задание**

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

#### **Входные данные:**

- $n$  — количество городов ( $5 \leq n \leq 15$ ).
- Матрица расстояний между городами размером  $n \times n$ , где  $graph[i][j]$  обозначает расстояние от города  $i$  до города  $j$ . Если  $graph[i][j]=0$  (и  $i \neq j$ ), это означает, что прямого пути между городами нет.

#### **Выходные данные:**

- Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.
- Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.
- Если такого пути не существует, вывести "no path".

## Выполнение работы

Программа реализует два алгоритма для решения задачи коммивояжера: точный алгоритм на основе динамического программирования с использованием битовых масок и приближённый эвристический алгоритм АЛШ-2. Оба алгоритма принимают на вход количество вершин  $n$  и матрицу смежности графа.

### Точный алгоритм

#### Основные структуры данных

- `graph` – двумерный массив размером  $n \times n$ , исходная матрица смежности. Элемент `graph[i][j]` содержит вес ребра из вершины  $i$  в вершину  $j$ . Значение 0 означает отсутствие прямого пути.
- `dp` – двумерный массив размером  $2^n \times n$ , используемый для запоминания состояний. Элемент `dp[mask][pos]` хранит минимальную стоимость маршрута, проходящего через множество вершин и заканчивающегося в вершине `pos`. Инициализируется значением бесконечности.
- `parent` – двумерный список предназначенный для восстановления оптимального пути. В `parent[mask][pos]` записывается индекс предыдущей вершины, из которой был совершен оптимальный переход в текущее состояние. Инициализируется значением -1.

#### Описание алгоритма

1. Базовый случай задается для стартовой вершины 0: `dp[1][0] = 0`. Битовая маска 1 означает, что посещена только нулевая вершина.
2. Запускается цикл по всем возможным маскам от 1 до  $2^n - 1$ .  
Оптимизация: маски, в которых не установлен младший бит (то есть не включающие стартовую вершину 0), пропускаются.
3. Для каждой валидной маски перебираются все входящие в неё вершины  $u$ . Если маршрут до  $u$  существует (`dp[mask][u] != INF`), алгоритм рассматривает переходы во все непосещенные вершины  $v$ . Если в матрице смежности есть ребро `graph[u][v] > 0`,

формируется новая маска  $\text{next\_mask} = \text{mask} \mid (1 \ll v)$  и вычисляется стоимость перехода:  $\text{new\_cost} = \text{dp}[\text{mask}][u] + \text{graph}[u][v]$ .

4. Если  $\text{new\_cost}$  оказывается меньше значения, уже хранящегося в  $\text{dp}[\text{next\_mask}][v]$ , происходит обновление стоимости пути, а в  $\text{parent}[\text{next\_mask}][v]$  сохраняется вершина  $u$ .
5. Замыкание цикла: После заполнения таблицы алгоритм перебирает все варианты возврата из последней посещенной вершины  $i$  (при полной маске) обратно в стартовую вершину 0. Выбирается вариант с минимальной итоговой суммой.
6. Восстановление пути: Начиная с последней вершины и полной маски, алгоритм с помощью массива  $\text{parent}$  раскручивает путь с конца, последовательно убирая посещенные вершины из маски операцией XOR. Полученный список разворачивается, и в конец добавляется вершина 0.

### **Оценка сложности**

Алгоритм использует таблицу мемоизации  $\text{dp}[\text{mask}][\text{pos}]$ , где  $\text{mask}$  — битовая маска посещённых вершин (всего  $2^n$  возможных состояний), а  $\text{pos}$  — индекс текущей вершины ( $n$  вариантов). Для каждого из  $n \cdot 2^n$  состояний выполняется цикл по всем  $n$  вершинам для проверки возможных переходов в непосещённые города. Таким образом, общее количество базовых операций пропорционально  $2^n \cdot n \cdot n$ . Следовательно, временная сложность  $O(n^2 \cdot 2^n)$ .

Для работы алгоритма требуется хранение двух таблиц:  $\text{dp}$  (для минимальных стоимостей путей) и  $\text{parent}$  (для восстановления итогового маршрута). Обе таблицы имеют размерность  $2^n \cdot n$ . Исходная матрица смежности занимает  $O(n^2)$  памяти, что асимптотически меньше, чем затраты на таблицы состояний. Итоговая пространственная сложность:  $O(n \cdot 2^n)$

### **Приближенный алгоритм**

#### **Основные структуры данных**

- `path` — одномерный список, хранящий последовательность вершин текущего строящегося маршрута.
- `unvisited` — множество (set), содержащее индексы вершин, которые еще не были включены в маршрут.

### Описание алгоритма

Алгоритм АЛШ-2 состоит из двух последовательных этапов: генерации стартового решения и его последующей оптимизации.

1. Жадный старт: Строго в соответствии с условием варианта построение маршрута начинается со стартовой вершины 0. На каждом шаге алгоритм просматривает смежные непосещенные вершины и выбирает инцидентное ребро с минимальным весом. Выбранная вершина добавляется в `path` и удаляется из множества `unvisited`. После обхода всех вершин маршрут принудительно замыкается добавлением перехода в вершину 0. Если на каком-то этапе дальнейший путь невозможен, алгоритм возвращает отсутствие решения.
2. Локальная оптимизация: Запускается цикл `while improved`, который продолжается до тех пор, пока в маршруте происходят улучшения.
3. Внутри цикла алгоритм перебирает все пары несмежных ребер  $(u_1, v_1)$  и  $(u_2, v_2)$  в текущем пути `path`.
4. Для каждой пары проверяется существование перекрестных ребер  $(u_1, u_2)$  и  $(v_1, v_2)$  в исходном графе. Если они существуют, вычисляется текущая суммарная длина разрываемых ребер и предполагаемая длина новых связей.
5. Если `new_dist < current_dist`, то выполняется перекоммутация: участок пути между  $v_1$  и  $u_2$  разворачивается с помощью среза `path[i:j+1] = reversed(path[i:j+1])`. Флаг `improved` устанавливается в значение `True`, и алгоритм уходит на новую итерацию поиска.

6. По завершении цикла (когда пересечений, ухудшающих маршрут, не осталось) вычисляется итоговая стоимость оптимизированного приближенного маршрута суммированием весов ребер.

### **Оценка сложности**

Работа алгоритма разбивается на два этапа. Построение начального маршрута жадным методом (поиск ближайшего соседа) требует  $n$  шагов, на каждом из которых просматривается до  $n$  смежных вершин. Это занимает  $O(n^2)$  операций.

На этапе локальной оптимизации алгоритм перебирает все возможные пары несмежных рёбер в текущем маршруте, что также требует  $O(n^2)$  операций на одну итерацию улучшения. Хотя теоретически максимальное количество итераций до достижения локального минимума может быть большим, на практике алгоритм быстро сходится, и общее время работы в среднем оценивается кубической зависимостью. Итоговая временная сложность: в среднем  $O(n^3)$ .

В отличие от точного метода, эвристический алгоритм не требует экспоненциальной памяти. Для его работы необходимо лишь хранение исходной матрицы весов графа размером  $n \times n$ , а также одномерных структур (списков и множеств) для текущего пути и перечня непосещённых вершин, которые занимают  $O(n)$  памяти. Итоговая пространственная сложность определяется размером исходной матрицы смежности и составляет  $O(n^2)$ .

## Тестирование

Тест 1

Ввод:

5

0 32 85 82 91

32 0 35 19 22

85 35 0 27 24

82 19 27 0 30

91 22 24 30 0

Вывод:

### РЕЗУЛЬТАТЫ

1. Точный метод (ДП):

Стоимость: 187

Маршрут: 0 -> 3 -> 2 -> 4 -> 1 -> 0

2. Приближенный метод (АЛШ-2):

Стоимость: 187

Маршрут: 0 -> 1 -> 4 -> 2 -> 3 -> 0

Тест 2

Ввод:

15

0 58 22 34 32 32 94 28 58 67 97 62 62 49 14

58 0 90 87 36 72 85 52 18 75 29 50 69 10 96

22 90 0 67 79 68 18 46 55 58 16 14 30 97 91

34 87 67 0 48 46 10 78 54 71 14 27 53 33 46

32 36 79 48 0 12 60 91 77 76 41 63 94 84 93

32 72 68 46 12 0 37 21 73 39 21 20 79 52 43

94 85 18 10 60 37 0 51 59 74 86 63 43 29 94

28 52 46 78 91 21 51 0 29 82 47 71 16 87 17

58 18 55 54 77 73 59 29 0 77 14 58 73 88 65  
67 75 58 71 76 39 74 82 77 0 69 13 95 96 94  
97 29 16 14 41 21 86 47 14 69 0 41 75 20 98  
62 50 14 27 63 20 63 71 58 13 41 0 49 21 83  
62 69 30 53 94 79 43 16 73 95 75 49 0 37 57  
49 10 97 33 84 52 29 87 88 96 20 21 37 0 80  
14 96 91 46 93 43 94 17 65 94 98 83 57 80 0

Вывод:

## РЕЗУЛЬТАТЫ

### 1. Точный метод (ДП):

Стоимость: 278

Маршрут: 0 -> 14 -> 7 -> 12 -> 13 -> 1 -> 8 -> 10 -> 3 -> 6 -> 2 -> 11 ->  
9 -> 5 -> 4 -> 0

### 2. Приближенный метод (АЛШ-2):

Стоимость: 301

Маршрут: 0 -> 14 -> 7 -> 12 -> 13 -> 11 -> 9 -> 5 -> 4 -> 1 -> 8 -> 10 ->  
3 -> 6 -> 2 -> 0

Тест 3

Ввод:

5

0 0 0 0 0

0 0 0 0 0

0 0 0 0 0

0 0 0 0 0

0 0 0 0 0

Вывод:

## РЕЗУЛЬТАТЫ

### 1. Точный метод (ДП):

no path



## 2. Приближенный метод (АЛШ-2):

no path

Тест 4

Ввод:

6

0 24 15 30 45 10

24 0 18 22 13 40

15 18 0 50 11 33

30 22 50 0 25 14

45 13 11 25 0 21

10 40 33 14 21 0

Вывод:

РЕЗУЛЬТАТЫ

### 1. Точный метод (ДП):

Стоимость: 85

Маршрут: 0 -> 5 -> 3 -> 1 -> 4 -> 2 -> 0

## 2. Приближенный метод (АЛШ-2):

Стоимость: 85

Маршрут: 0 -> 5 -> 3 -> 1 -> 4 -> 2 -> 0

Тест 5

Ввод:

5

0 15 0 0 20

15 0 10 0 0

0 10 0 25 0

0 0 25 0 30

20 0 0 30 0

Вывод:

РЕЗУЛЬТАТЫ

1. Точный метод (ДП):

Стоимость: 100

Маршрут: 0 -> 4 -> 3 -> 2 -> 1 -> 0

2. Приближенный метод (АЛШ-2):

Стоимость: 100

Маршрут: 0 -> 1 -> 2 -> 3 -> 4 -> 0

## **Вывод**

В ходе выполнения лабораторной работы была успешно решена задача коммивояжёра. Для её решения были реализованы и проанализированы два алгоритма: точный метод на основе динамического программирования и приближённый метод локального поиска (АЛШ-2).

Точный алгоритм гарантированно находит абсолютно кратчайший маршрут, однако из-за высокой вычислительной сложности  $O(n^2 \cdot 2^n)$  его применение ограничено графами небольшого размера (до 15 вершин).

Приближённый алгоритм АЛШ-2, напротив, обладает высокой производительностью и работает за полиномиальное время  $\approx O(n^3)$ . Хотя он не гарантирует нахождения идеального пути и может выдавать субоптимальный маршрут с небольшой погрешностью, он позволяет практически мгновенно получать хорошие результаты для графов большой размерности.

Таким образом, цель работы достигнута: алгоритмы реализованы корректно и наглядно демонстрируют фундаментальный компромисс между скоростью вычислений и абсолютной точностью результата.